Learn
Practice
Build
Deploy
Grow !

Day 1: Docker Basics

Build, Ship and Run Applications Anywhere !

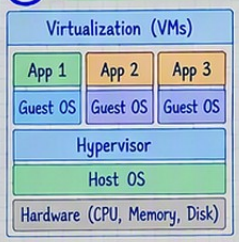
Containers Make Development Simple !

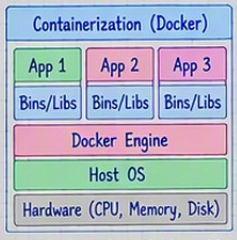
1 What is Docker ?

- Docker is an open-source platform to build, ship and run applications in containers.
- A container packages the application and all its dependencies (libraries, config, runtime) together.
- Provides consistent environment across development, testing and production.

"Build once, run anywhere !"

2 Containerization vs Virtualization





- Each VM has its own OS.
- Heavy and resource intensive.
- Slower to start.
- Suitable for running different OS.

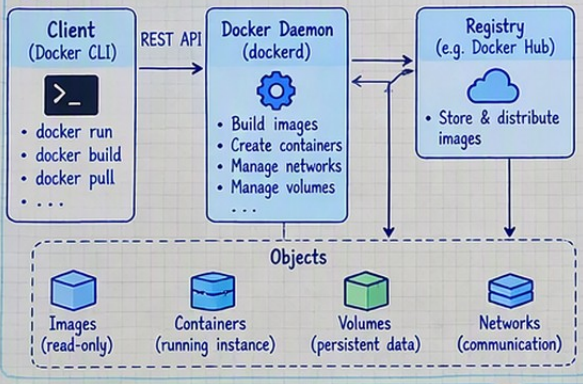
- Containers share the host OS.
- Lightweight and fast.
- Uses fewer resources.
- Ideal for microservices and modern applications.

3 Docker vs Virtual Machine

Feature	Docker (Container)	Virtual Machine
OS	Shared (host OS)	Own OS per VM
Size	MBs	GBs
Start time	Seconds	Minutes
Resource usage	Low	High
Isolation	Process level	Full OS level
Portability	High	Medium
Use case	Microservices, CI/CD	Legacy apps, different OS

Containers are lightweight, but VMs provide stronger isolation.

4 Docker Architecture



5 What is Docker Engine ?

- Docker Engine consists of:
  - Docker Daemon (dockerd)
  - REST API
  - Docker CLI (client)
- Responsible for building images, running containers, managing networks and volumes.
- Runs natively on Linux, Windows and macOS.

Docker Engine = Core of Docker !

6 Docker Registries

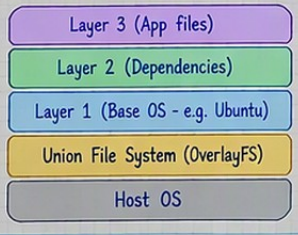
- A registry stores Docker images.
- Public: Docker Hub (default)
- Private/Cloud:
  - Amazon ECR (AWS)
  - Azure ACR (Azure)
  - Google GCR (Google Cloud)
- Self-hosted: Harbor, Nexus, etc.

Push your images to a registry to share and deploy !

Docker Hub · Amazon ECR · Azure ACR · Google GCR

7 Union File System (AUFS / OverlayFS)

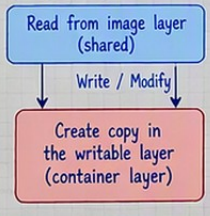
- Docker uses Union File System to build images in layers.
- Each layer is read-only and stacked on top of each other.
- Common drivers: AUFS (older), OverlayFS (default in modern Linux).



Each layer only stores the changes, not the entire filesystem !

8 Copy-on-Write (CoW)

- Containers share the same image layers.
- When a file is changed, a new copy is written to a writable layer (CoW).
- Original layer remains unchanged.



CoW saves disk space and makes containers fast !

9 Why is Docker Lightweight ?

- Shares host OS kernel.
- No need for separate OS per container.
- Uses Union File System and CoW.
- Only application and its dependencies are packaged.
- Leverages Linux namespaces and cgroups for isolation and resource control.

Less overhead = Faster, smaller, efficient !

10 What happens when you run `docker run` ?

1. Docker CLI sends request to Docker Daemon.
2. Daemon checks if the image is available locally.
3. If not, pulls the image from a registry (e.g. Docker Hub).
4. Creates a writable container layer using CoW.
5. Sets up networking (assigns IP, connects to network).
6. Mounts volumes (if specified).
7. Applies configuration (env variables, ports, etc.).
8. Starts the container and runs the specified command (e.g. CMD or ENTRYPOINT).

11 Common Commands

Command	Purpose	Example
docker run	Run a container	docker run nginx
docker ps	List running containers	docker ps
docker images	List images	docker images
docker pull	Pull image from registry	docker pull nginx
docker build	Build image from Dockerfile	docker build -t myapp .
docker exec	Access running container	docker exec -it <id> bash
docker logs	View container logs	docker logs <id>
docker rm	Remove container	docker rm <id>
docker rmi	Remove image	docker rmi <image>

12 Interview Tips

- [x] What is Docker and why is it used?
- [x] Explain containerization vs virtualization.
- [x] How does Docker architecture work?
- [x] What is a Docker image vs container?
- [x] What is a registry? Name some examples.
- [x] Explain Union File System and CoW.
- [x] Why is Docker lightweight?
- [x] What happens internally when you run `docker run` ?

Practice commands and real scenarios to build confidence !

Containers standardize, simplify and accelerate application delivery !

Learn | Practice | Build | Deploy | Grow

Learn
Practice
Build
Deploy
Grow !

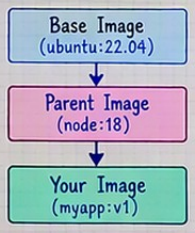
Day 2: Docker Images & Dockerfile

Build Efficient Images · Understand Layers · Write Better Dockerfiles

Write
Build
Ship
Repeat !

1 Base Image vs Parent Image

- Base image: starting point for your image (e.g. ubuntu, alpine, python).
- Parent image: image specified in FROM in your Dockerfile (can be a base image or a custom image).
- Every image (except scratch) has a parent.



Your image is built on top of parent image which is built on top of base image.

All base images are parent images, but not all parent images are base images !

2 What is a Dockerfile ?

- A Dockerfile is a text file with a set of instructions to build a Docker image.
- It defines the base image, dependencies, files to copy, commands to run etc.
- Version control your Dockerfile (keep it in Git).

Code Your Image !

Example: Simple Dockerfile

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .
# Expose port
EXPOSE 8000
# Start the application
CMD ["python", "app.py"]
```

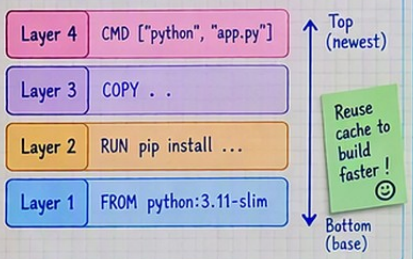
3 Dockerfile Instructions

Instruction	Purpose (What it does)
FROM	Sets the base/parent image
RUN	Runs commands during build
CMD	Default command to run container
ENTRYPOINT	Configures the main executable
EXPOSE	Documents the port the container listens on
ENV	Sets environment variables
WORKDIR	Sets the working directory
ADD	Copies files (supports URL & auto-extract)
COPY	Copies files from host to image (preferred)
VOLUME	Creates a mount point for volumes
USER	Sets user to run the container
LABEL	Adds metadata to the image

Use only the instructions you need. Keep it simple !

4 Image Layering (Build Layers)

- Each instruction creates a new read-only layer.
- Layers are cached to speed up future builds.
- If a layer changes, all subsequent layers are rebuilt.



Reuse cache to build faster !

5 CMD vs ENTRYPOINT

- CMD: provides default command (can be overridden at runtime).
- ENTRYPOINT: sets the main executable (always runs, arguments can be passed).

CMD Example

```
FROM ubuntu
CMD ["echo", "Hello"]

# Can be overridden:
docker run image ls
```

ENTRYPOINT Example

```
FROM ubuntu
ENTRYPOINT ["echo"]

# Always runs:
docker run image "Hi"
```

Use ENTRYPOINT for main apps.
Use CMD for default args !

6 ADD vs COPY

- COPY: only copies files (recommended).
- ADD: can copy files, URLs and can auto-extract archives (tar, zip).
- Use COPY in most cases for better control.

COPY Example

```
COPY app.py /app/
```

- Simple and predictable
- Recommended

ADD Example

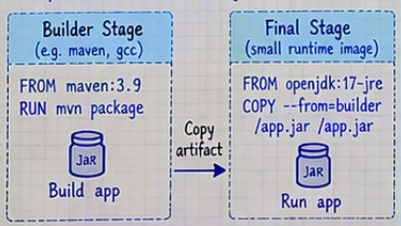
```
ADD https://example.com/
file.tar.gz /app/
```

- Supports URL
- Auto extracts archives
- Use only when needed

Prefer COPY unless you really need ADD features !

7 Multi-stage Build

- Use multiple FROM stages to build smaller images.
- Build dependencies in a builder stage and copy only required artifacts to final image.



Smaller images, faster deployment, more secure !

8 Reduce Image Size (Tips)

- ☑ Use slim or alpine base images.
- ☑ Use multi-stage builds.
- ☑ Remove unnecessary packages (apt-get clean).
- ☑ Use .dockerignore to exclude files.
- ☑ Combine RUN commands.
- ☑ Avoid copying large files.
- ☑ Use specific version tags (e.g. python:3.11-slim).
- ☑ Remove build tools and cache.

Smaller images
= Faster builds
= Lower cost
= Better security

9 Useful Commands

Command	Purpose
docker image ls	List local images
docker image inspect <img>	Show image details
docker image prune	Remove unused images
docker build -t name:tag .	Build image from Dockerfile
docker pull name:tag	Pull image from registry
docker push name:tag	Push image to registry
docker rmi <image>	Remove image
docker history <image>	Show image layers
docker tag <src> <dst>	Tag an image

Clean unused images regularly !

10 Tagged Images

- Tags give a name and version to an image.
- Format: <name>:<tag>
- Examples: nginx:latest, myapp:v1, python:3.11
- Helps to manage different versions.

Example:

```
docker build -t myapp:v1 .
docker tag myapp:v1 myapp:v2
docker push myapp:v1
```

Use meaningful tags (e.g. v1.0.0) instead of just latest !

11 Common Scenarios & Solutions

Scenario	Solution
Image size too large	Use slim base image, multi-stage build, clean packages, .dockerignore
Build slows pipeline	Use build cache, reorder layers, copy requirements first
Python missing dependencies	Ensure requirements.txt is copied and run pip install. Check virtual env and correct base image

Always check build logs — they give useful hints !

12 Interview Tips

- ☑ Explain difference between base and parent image.
- ☑ What is each Dockerfile instruction used for?
- ☑ CMD vs ENTRYPOINT (with examples).
- ☑ ADD vs COPY — when to use which?
- ☑ Why and how to use multi-stage builds?
- ☑ How to reduce image size?
- ☑ How does image layering and cache work?
- ☑ Explain useful docker image commands.
- ☑ Why are tags important?

Practice writing Dockerfiles for real applications !

Build Better Images → Deploy Faster → Run Anywhere !

Learn
Practice
Build
Troubleshoot
Be Confident !

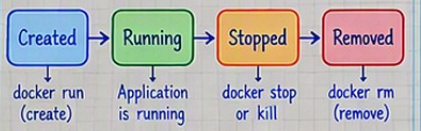
Day 3: Docker Containers

Build · Ship · Run · Manage · Anywhere

Containers make apps portable, consistent and scalable !

1 What is a Container Lifecycle ?

- A container goes through different states in its lifecycle.



Containers are ephemeral by default. Use volumes to persist data !

2 docker stop vs docker kill

- docker stop sends SIGTERM and waits (default 10s) before SIGKILL.
- Allows graceful shutdown.
- docker kill sends SIGKILL immediately (force stop).
- Use kill only when container is not responding.

Command	Signal	Behavier	Use case
docker stop	SIGTERM (then SIGKILL)	Graceful shutdown	Normal stop
docker kill	SIGKILL	Force stop (immediate)	When unresponsive

3 Common Docker Commands

Command	Description	Example
docker ps	List running containers	docker ps
docker ps -a	List all containers	docker ps -a
docker run	Create and start container	docker run -d nginx
docker exec	Run command in container	docker exec -it <id> bash
docker logs	View container logs	docker logs -f <id>
docker rm	Remove container	docker rm <id>
docker start	Start stopped container	docker start <id>
docker inspect	Detailed info (JSON)	docker inspect <id>

4 Detached vs Interactive Mode

Detached mode (-d)

- Runs container in background.
- Returns container ID.
- Useful for servers and services.
- Example:

```
docker run -d nginx
```

View logs:

```
docker logs -f <id>
```

Interactive mode (-it)

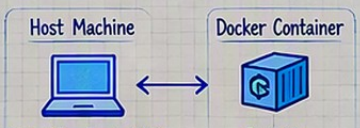
- Keeps STDIN open and allocates a terminal.
- Useful for bash, debugging, manual work.
- Example:

```
docker run -it ubuntu bash
```

Use -it when you need a shell inside the container!

5 Ports: Host Port vs Container Port

- Use -p to map a host port to a container port.
- Format: -p <host_port>:<container_port>



http://localhost:8080 (Host Port 8080)

Port 80 (Container Port)

Example:

```
docker run -d -p 8080:80 nginx
```

Now you can access nginx on http://localhost:8080

6 Restart Policies

- Define what happens when a container exits.

Policy	Behavior	Use case
no	Do not restart	Default
always	Always restart	Keep critical services running
on-failure	Restart only on non-zero exit	For apps that may fail occasionally
unless-stopped	Always restart unless manually stopped	Useful for production

Set policy using --restart

```
docker run --restart always nginx
```

7 Shell Access Inside a Container

- Use docker exec to run a command in a running container.
- Common use case: get a shell.
- Example:

```
docker exec -it <container_id> bash
```

Use exec to debug running containers !

Run a single command:

```
docker exec <id> ls -l /app
```

8 Copy Files Between Host and Container

- Copy from host to container:

```
docker cp ./app.py <container_id>:/app/
```

- Copy from container to host:

```
docker cp <container_id>:/app/logs.txt ./logs.txt
```



Useful for copying logs, configs or files for debugging !

9 Troubleshooting Scenarios

1 Container restarts again and again

- Check logs: docker logs <id>
- Check exit code: docker inspect <id>
- Look for app errors or missing config.

2 Container running but app not reachable

- Check port mapping: docker ps
- Verify app is listening inside container.
- Check firewall/security groups.

3 Container logs not showing

- Use docker logs -f <id>
- Check logging driver configuration.
- Ensure app writes to stdout/stderr.

Check logs first ! Logs tell the real story !

10 Practical Example

Run a web server, check, stop and remove it.

```
# 1. Run nginx
docker run -d -p 8080:80 --name mynginx nginx

# 2. Check running container
docker ps

# 3. Access in browser
http://localhost:8080

# 4. Stop container
docker stop mynginx

# 5. Remove container
docker rm mynginx
```

Try this hands-on ! Practice makes perfect !

11 Quick Reference Summary

- [x] Understand container lifecycle.
- [x] Know docker stop vs docker kill.
- [x] Use -d for detached, -it for interactive.
- [x] Map ports using -p.
- [x] Use restart policies for reliability.
- [x] Use docker exec for shell access.
- [x] Use docker cp to copy files.
- [x] Troubleshoot using logs and inspect.
- [x] Practice with real examples.
- [x] Clean up unused containers and images.

Small commands Big control !

12 Interview Questions

- [x] What is a container? How is it different from a VM?
- [x] Explain the container lifecycle.
- [x] Difference between docker stop and docker kill?
- [x] How do you expose a container port to the host?
- [x] What is the purpose of --restart policy?
- [x] How do you access a container shell?
- [x] How do you copy files between host and container?
- [x] What would you do if a container keeps restarting?
- [x] How do you check container logs?
- [x] Why are containers lightweight?
- [x] What happens internally when you run docker run?
- [x] Give a real-world example of using Docker.

"Containers package your app, but good knowledge runs your career !"

Build Skills Today → Ship Better Tomorrow !

Build
Run !
Persist
Scale !

Day 4: Docker Volumes & Storage

Keep your data safe – Containers are ephemeral, Volumes are not !

Containers may come and go but your data should stay !

Learn
Practice
Troubleshoot
Grow !

1 Why Persistent Storage Matters ?

- Containers are ephemeral (temporary).
- When a container is deleted or recreated, its writable layer is lost.
- Any data stored inside the container (e.g. logs, database files) will be lost unless you use a volume.



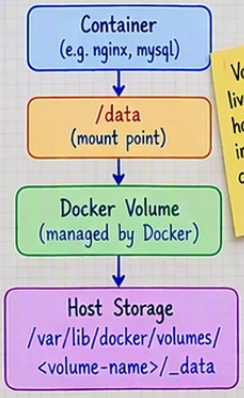
"Use volumes to persist important data outside the container lifecycle."

2 Types of Docker Volumes

Type	Description	Use Case	Example
Named Volumes	Managed by Docker (stored in Docker's volume directory)	Preferred for persistent data (e.g. databases)	-v mydata:/data or docker volume create mydata
Bind Mounts	Maps a host directory to a container directory	Useful for development or custom paths	-v /host/path: /container/path
Tmpfs Mounts	Stored in host memory (not on disk)	Temporary data (sensitive data, caching)	--tmpfs /tmp

Use Named Volumes for production, Bind Mounts for development and Tmpfs for temporary data.

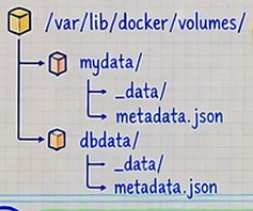
3 Volume Architecture



Volume lives on the host, not inside the container !

4 Volumes on Linux Host

- Named volumes are stored under:
  `/var/lib/docker/volumes/`
- Each volume has a directory with _data folder.
- Docker manages this location automatically.



Don't modify volume data directly unless needed !

5 Storage Driver Role

- Storage drivers (e.g. overlay2) manage how container filesystem layers and volumes are stored on disk.
- Volumes are separate from container writable layers.
- Common storage drivers: overlay2 (default), aufs, devicemapper, btrfs, zfs.

Volumes are stored outside the container layers and are not affected by image or container deletion.

6 Useful Docker Volume Commands

Command	Description
docker volume create <name>	Create a named volume
docker volume ls	List all volumes
docker volume inspect <name>	Show details (mountpoint, etc.)
docker volume rm <name>	Remove a volume
docker volume prune	Remove unused volumes

Example:

```
$ docker volume create dbdata
$ docker volume ls
$ docker volume inspect dbdata
$ docker volume rm dbdata
```

Use "docker volume inspect" to find the actual host path !

7 Backup and Restore Volumes

- You can backup a volume using tar or rsync.
- Use a temporary container to access the volume.

Backup using tar

```
$ docker run --rm -v mydata:/data -v $(pwd):/backup \
  alpine tar czf /backup/mydata.tar.gz -C /data .
```

Restore using tar

```
$ docker run --rm -v mydata:/data -v $(pwd):/backup \
  alpine tar xzf /backup/mydata.tar.gz -C /data
```

Using rsync (from container to host)

```
$ docker run --rm -v mydata:/data -v $(pwd):/backup \
  alpine sh -c "apk add --no-cache rsync && \
  rsync -av /data/ /backup/"
```

Backup regularly, especially for production data !

8 Common Scenarios

1 Database loses data after restart

- Reason: Data stored inside container writable layer.
- Solution: Use a named volume.

```
$ docker run -d --name mysql \
  -v dbdata:/var/lib/mysql \
  mysql:latest
```

2 Two containers need shared data

- Use the same named volume for both.

```
$ docker run -d --name app1 -v shared:/data ...
$ docker run -d --name app2 -v shared:/data ...
```

3 App needs read-only config

- Use a bind mount with read-only flag.

```
$ docker run -d -v /host/config:/app/config:ro \
  nginx:latest
```

9 Practical Example: Nginx with Volume

Create and run an Nginx container with a named volume.

```
$ docker volume create html_data
$ docker run -d --name web \
  -p 8080:80 \
  -v html_data:/usr/share/nginx/html \
  nginx:latest
```

*Same volume can be reused by multiple containers !



Update files in the volume and refresh the browser to see the changes.

10 Troubleshooting Tips

Issue	Possible Cause	Solution
Data lost after restart	No volume used	Use named volume or bind mount
Permission denied	Wrong file permissions	Use correct UID/GID or chmod
Volume not found	Wrong volume name	Check with docker volume ls
No space left on device	Disk full	Clean unused volumes (docker volume prune)
Changes not reflected	Using image layer	Use bind mount or volume correctly

Check volume mount path and permissions when facing issues !

11 Interview Tips

- [x] What is a Docker volume?
- [x] Difference between volumes and bind mounts?
- [x] Where are volumes stored on the host?
- [x] How do you backup and restore a volume?
- [x] What is tmpfs and when would you use it?
- [x] How do you share data between containers?
- [x] How do you make a volume read-only?
- [x] What happens to volumes when a container is deleted?

12 Key Takeaways

- [x] Containers are ephemeral.
- [x] Use volumes for persistent data.
- [x] Named volumes are managed by Docker.
- [x] Bind mounts are useful for development.
- [x] Tmpfs is stored in memory.
- [x] Always backup important data.
- [x] Understand storage drivers.
- [x] Volumes can be shared across containers.
- [x] Use read-only mounts for sensitive config.
- [x] Practice with real-world scenarios.

"Persist your data, Protect your applications, Build reliable systems !"

Docker Today → Better Applications Tomorrow !

Learn
Practice
Troubleshoot
Build Real Skills

Day 5: Docker Networking

Connect Containers · Enable Communication · Understand Network Drivers

Good Networking = Scalable Apps!

1 Docker Networking Basics

- Docker provides isolated networks for containers to communicate with each other and with external world.
- Each container gets a network interface (eth0) and an IP address.
- Docker uses network drivers to manage networking (bridge, host, none, overlay, etc).
- By default, containers can communicate with containers on the same user-defined bridge network using container names (DNS).

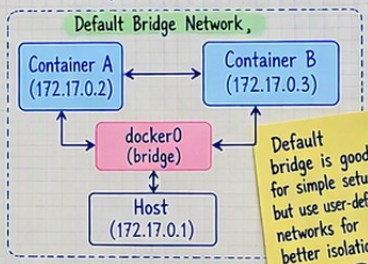
"Networking is the glue that connects your containers!"

2 Network Drivers (Types)

Driver	Description	Use Case
bridge (default)	Creates an isolated network on a single host using a Linux bridge (eg. docker0)	Single host containers (most common)
host	Container shares the host's network namespace	High performance (doesn't get own IP)
none	No networking (container has no network interface)	Security, isolated containers
overlay	Multi-host network (using Docker Swarm)	Containers across multiple hosts
macvlan	Assigns MAC address to container (appears as physical device)	Legacy apps, network integration

3 Default Bridge Network

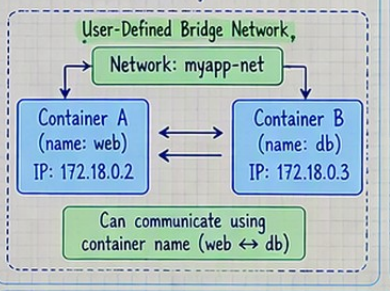
- When you install Docker, a default bridge network (docker0) is created.
- Containers started without specifying a network are attached to this bridge network.
- Containers can communicate using IP but not by container name (DNS resolution works only in user-defined networks).



Default bridge is good for simple setups but use user-defined networks for better isolation!

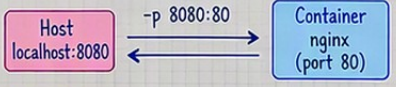
4 User-Defined Bridge Network

- Provides automatic DNS resolution (use container names instead of IP).
- Better isolation and management.
- Containers on the same user-defined network can communicate easily.



5 Port Publishing (Host ↔ Container)

- By default, containers are isolated and not accessible from outside.
- Use -p or --publish to map a host port to a container port.
- Format: -p <host_port>:<container_port>



Example:

```
$ docker run -d -p 8080:80 --name web nginx
```

- Access in browser: http://localhost:8080
- You can also use -p 127.0.0.1:8080:80 (to bind only to localhost)

Use -P to publish all EXPOSEd ports to random host ports!

6 Embedded DNS in Docker

- User-defined networks provide an embedded DNS server.
- Containers can resolve each other by name (no need to hardcode IPs).
- Works across all containers in the same network.

Example: Container A can ping Container B

```
$ docker run -d --name db --network myapp-net mysql
$ docker run -it --network myapp-net busybox
# inside busybox container
/ # ping db
```

Use container names, not IPs!

7 Connect a Container to a Network

- You can connect an existing container to a network using docker network connect.

Example:

```
$ docker network create myapp-net
$ docker run -d --name app1
  --network myapp-net nginx
$ docker network connect myapp-net app2
```

A container can be connected to multiple networks!

8 Useful Docker Network Commands

Command	Purpose
docker network ls	List all networks
docker network create <name>	Create a new network
docker network inspect <name>	Inspect network details
docker network prune	Remove unused networks
docker network connect <network> <container>	Connect container to network
docker network disconnect <network> <container>	Disconnect container from network

9 Common Scenarios & Solutions

1 Containers cannot communicate
- Check if both containers are on the same network.
- Use docker network inspect to verify.
- Use container names (not IPs).

2 App runs locally but not via Docker
- Check if the port is exposed and published.
- Verify the app is listening on 0.0.0.0 (not 127.0.0.1).
- Check firewall / security groups (if on cloud).

3 Containers on separate networks need to communicate
- Connect them to a common network.
- Or use multiple networks (docker network connect).
- For multi-host: use overlay network (Swarm).

10 Troubleshooting Network Issues (Step by Step)

1	Check network membership	docker network inspect <network>
2	Check DNS resolution	ping <container-name> / nslookup
3	Check IP connectivity	ping <container-ip>
4	Check if application is listening	netstat -tulnp (inside container)
5	Check port publishing	docker ps (verify PORTS)
6	Check firewall / security groups	ufw status / cloud SG rules

Follow a systematic approach to find the root cause!

11 Interview Tips

- [x] Explain different network drivers.
- [x] When to use bridge vs host vs overlay?
- [x] How does Docker DNS work?
- [x] How to connect containers on different networks?
- [x] How do you troubleshoot container connectivity issues?
- [x] What happens when you use -p?
- [x] Can containers communicate across hosts?
- [x] How would you secure Docker networking?
- [x] Real-world scenario discussion.

12 Key Takeaways

- [x] Docker provides multiple network drivers.
- [x] Use user-defined networks for better control.
- [x] Use container names (DNS) instead of IPs.
- [x] Port publishing exposes containers to outside.
- [x] Use overlay network for multi-host setups.
- [x] Always verify network settings during troubleshooting.

Well connected containers build powerful applications!

Containers + Networking = Endless Possibilities!

Learn | Practice | Troubleshoot | Grow

Learn
Practice
Build
Deploy
Grow !

Day 6: Docker Compose

Manage Multi-Container Applications Easily !

Docker Compose

One command to run your entire stack !

1 What is Docker Compose ?

- Docker Compose is a tool to define and run multi-container applications using a single YAML file (compose.yaml).
- It helps to manage services, networks and volumes together.
- Instead of running multiple docker run commands, we define everything in one file and start with a single command.
- Great for development, testing and local environments.

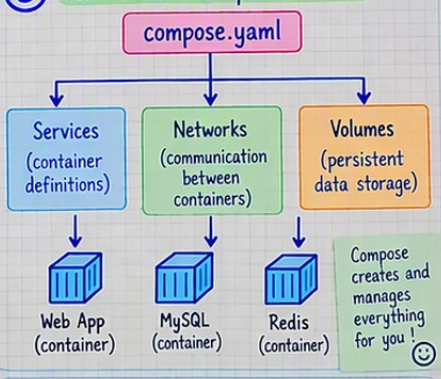
Compose = Define → Run → Manage multi-container apps !

2 Why use Compose ?

- ☑ Simplifies multi-container setup.
- ☑ Defines infrastructure as code (YAML).
- ☑ Manages networks, volumes, environment variables.
- ☑ Easy to start, stop, scale and view logs.
- ☑ Keeps configurations consistent.
- ☑ Useful for local development, CI/CD, testing.

Single command to run your entire application !

3 How Docker Compose Works ?



Compose creates and manages everything for you !

4 compose.yaml Example (Web + MySQL)

```yaml
version: '3.8'
services:
  web:
    build: ./app
    container_name: web
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=mysql
      - DB_USER=${MYSQL_USER}
      - DB_PASSWORD=${MYSQL_PASSWORD}
    depends_on:
      - mysql
    networks:
      - appnet

  mysql:
    image: mysql:8
    container_name: mysql
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=appdb
      - MYSQL_USER=appuser
      - MYSQL_PASSWORD=apppassword
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD","mysqladmin","ping","-h","localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - appnet

volumes:
  db_data:

networks:
  appnet:
```

Define your entire application in one file !

Real example: Web App + MySQL with persistent storage !

5 Key Sections in Compose

services:	Defines containers (name, image/build, ports, environment, etc.)
volumes:	Defines named volumes for persistent storage.
networks:	Defines networks for communication between containers.
depends_on:	Specifies startup order (does not wait for readiness).
env_file:	Load environment variables from a file.
healthcheck:	Check if a container is healthy.
restart:	Restart policy (always, on-failure, etc.)

6 Useful Docker Compose Commands

Command	Description
docker compose up	Start all services
docker compose up -d	Start in detached mode
docker compose down	Stop and remove services
docker compose logs	View logs of services
docker compose ps	List running services
docker compose exec <svc> <cmd>	Run command in a service
docker compose config	Validate and view the compose file
docker compose restart	Restart services
docker compose up --scale web=3	Scale a service

Note: Older versions use docker-compose (with hyphen).

```
docker-compose up      # legacy
docker compose up      # current (v2)
```

Use docker compose (cloud-native plugin) !

7 Environment Variables

- Use environment: with key-value pairs or use env_file to load from a file.
- Syntax: ${VAR} to use shell variables.

Using env_file:

```yaml
services:
  web:
    env_file:
      - .env
```

Using $(VAR):

```yaml
environment:
  - DB_HOST=${DB_HOST}
  - DB_USER=${DB_USER}
```

Keep sensitive values in .env file and do not commit to Git !

8 Healthcheck

- Used to check if a container is healthy.
- Helps other services (or orchestrators) know when the service is ready.
- Example (for MySQL):

```yaml
healthcheck:
  test: ["CMD","mysqladmin","ping","-h","localhost"]
  intereal: 10s
  timeout: 5s
  retries: 5
```

Healthcheck ensures service is actually ready !

9 Scaling Services

- Scale multiple instances of a service.
- Command: docker compose up --scale web=3
- Creates 3 containers for the web service.

Useful for load testing or local scaling !

10 Common Scenarios & Troubleshooting

Scenario	Possible Cause	Fix
MySQL starts slowly and Node.js fails	depends_on does not wait for readiness	Use healthcheck and wait-for-it script or retry logic in app
Environment variables not loading	.env file missing or wrong path	Check file location, use env_file, verify $(VAR)
Port already in use	Host port conflict	Change host port or stop existing service
Containers cannot communicate	Not on same network	Use a user-defined network
Data not persisting	Volume not configured	Use named volume and mount it
Changes not reflected	Image not rebuilt	Run docker compose up --build

Check logs:
docker compose logs <service>
It gives the real clue !

11 Interview Tips

- ☑ What is Docker Compose and why is it used?
- ☑ Explain services, networks and volumes in Compose.
- ☑ How do you pass environment variables?
- ☑ What is the difference between depends_on and healthcheck?
- ☑ How do you scale a service?
- ☑ Explain the compose.yaml structure.
- ☑ How do you troubleshoot a multi-container app?
- ☑ When would you use Compose vs Kubernetes?
- ☑ Real-world example of using Docker Compose.

Be ready with real examples !

12 Key Takeaways

- ☑ Docker Compose simplifies multi-container apps.
- ☑ Use YAML to define services, networks, volumese.
- ☑ Environment variables and .env files are important.
- ☑ Healthcheck ensures service readiness.
- ☑ Use named volumes for persistent data.
- ☑ Use custom networks for container communication.
- ☑ Compose is ideal for development and testing.
- ☑ Know common commands and troubleshooting steps.
- ☑ Same concepts apply in Kubernetes (but Compose is for local/dev).

Define Once . Run Everything . Build Faster !

Learn
Practice
Secure
Build
Grow !

Day 7: Docker Security

Build Secure Containers → Protect Images → Follow Best Practices

Security is not a feature It's a mindset !

1 Run as Non-Root User

- Running containers as root is a security risk.
- Use a non-root user with USER in Dockerfile.
- Helps limit the impact if container is compromised.

Dockerfile (example)

```
FROM ubuntu:22.04
RUN useradd -m appuser
USER appuser
WORKDIR /app
COPY . .
CMD ["python", "app.py"]
```

Never run containers as root unless absolutely required !

2 Image Signing & Provenance

- Signed images ensure integrity and trust.
- Docker Content Trust (DCT) uses Notary to sign and verify images.
- Helps prevent tampered or malicious images.



Ensures the image is from a trusted source and not modified !

3 Docker Content Trust (DCT)

- Uses Notary (The Update Framework).
- Signs images with a private key.
- Verifies with a public key during pull.
- Enable using environment variable:

```
export DOCKER_CONTENT_TRUST=1
```

Example

```
$ export DOCKER_CONTENT_TRUST=1
$ docker pull myapp:1.0
$ docker push myapp:1.0
```

4 Seccomp (Syscall Filtering)

- seccomp restricts the system calls a container can make.
- Helps reduce the attack surface.
- Use default profile or a custom

Block unnecessary system calls !

Run with default seccomp profile:

```
$ docker run --security-opt \
  seccomp=default.json myapp
```

5 Linux Capabilities

- Linux capabilities provide fine-grained privileges instead of full root access.
- Use --cap-add to add specific capabilities.
- Use --cap-drop to remove unnecessary ones.

Examples:

```
# Add a capability (e.g. NET_ADMIN)
$ docker run --cap-add=NET_ADMIN myapp

# Drop all and add minimal needed
$ docker run --cap-drop=ALL \
  --cap-add=NET_BIND_SERVICE myapp
```

Follow the principle of least privilege !

6 Docker Daemon Security

- Docker daemon (dockerd) has high privileges on the host.
- Protect daemon with proper access controls.
- Avoid exposing it over an untrusted network.
- Use TLS for remote access.
- Limit who can run docker commands (use sudo/docker group carefully).

Daemon = High risk Keep it secured !

7 Avoid Mounting Docker Socket

- /var/run/docker.sock gives full control over the host to the container.
- If a container gets access to the socket, it can run privileged containers, access host files, etc.
- Avoid mounting the Docker socket unless absolutely needed.

⚠ Do NOT mount /var/run/docker.sock into containers. It is equivalent to root access on the host !

8 Security Hardening Checklist

- ☑ Use minimal base image (e.g. alpine).
- ☑ Run as non-root user.
- ☑ Scan images for vulnerabilities.
- ☑ Pin versions / use image digests.
- ☑ Remove unnecessary packages and tools.
- ☑ Use read-only filesystem when possible.
- ☑ Use secrets outside the image (e.g. Docker secrets, KMS, environment variables, not hardcoded).
- ☑ Limit capabilities (drop all, add only required).
- ☑ Use resource limits (CPU, Memory).
- ☑ Keep Docker and base images updated.

A more secure image = A safer production environment !

9 Practical Examples

Read-only filesystem:

```
$ docker run --read-only myapp
```

Use tmpfs for writable directories:

```
$ docker run --read-only \
  --tmpfs /tmp myapp
```

Run as non-root with limited caps:

```
$ docker run -u 1000:1000 \
 --cap-drop=ALL --cap-add=NET_BIND_SERVICE
  myapp
```

10 Real-World Scenarios

1 Security audit fails

- Check if running as root.
- Verify image is signed and from trusted source.
- Scan image for vulnerabilities (e.g. Trivy, Docker Scout).
- Ensure sensitive data is not in image.

2 Container has too many permissions

- Review capabilities (docker inspect).
- Use --cap-drop=ALL and add only required.
- Check for privileged mode (--privileged) and avoid it.

3 Malware found in image

- Use image scanning tools (Trivy, Clair, Docker Scout).
- Pull from trusted registries.
- Verify image signature (DCT).
- Rebuild image from clean base.

Detect Respond Fix Prevent !

11 Threat → Control Mapping

Threat / Risk	Control / Mitigation
Running as root	Use non-root USER
Tampered image	Use image signing (DCT)
Malicious image	Scan images (Trivy, Clair)
Excessive permissions	Use --cap-drop, least privilege
Container breakout	Use seccomp, AppArmor, minimal image
Docker socket exposed	Do NOT mount /var/run/docker.sock
Sensitive data in image	Use secrets (not in image)
Untrusted registry	Use trusted registries, verify signatures
Vulnerable base image	Keep images updated, scan regularly

12 Interview Tips

- ☑ Why is it recommended to run containers as non-root?
- ☑ What is Docker Content Trust?
- ☑ How does seccomp improve security?
- ☑ What are Linux capabilities? Give examples.
- ☑ Why should we avoid mounting Docker socket?
- ☑ How would you secure a Docker daemon?
- ☑ How do you scan a Docker image for vulnerabilities?
- ☑ What is image provenance and why is it important?
- ☑ What steps would you take after finding malware in an image?

Security is everyone's responsibility — Build. Scan. Run. Secure !

"A secure container today, a safer tomorrow !"

Learn | Practice | Secure | Grow

Learn
Practice
Troubleshoot
Grow !

Day 8: Docker Logging & Monitoring

See what's happening • Find issues early • Keep your containers healthy

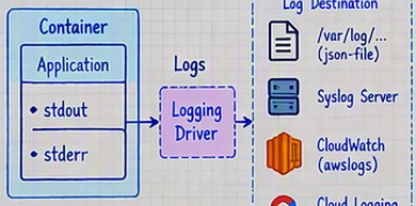
Logs tell the story ! Monitoring keeps you in control !

1 Docker Logging Drivers

- Docker uses logging drivers to capture container stdout and stderr.
- Different drivers send logs to different backends (file, syslog, cloud, etc.).
- Default driver is json-file.

Driver	Description	Use Case
json-file	Stores logs in JSON format on local disk	Default, simple setup
syslog	Sends logs to syslog server	Centralized logging
fluentd	Sends logs to fluentd collector	ELK / Fluentd stack
awslogs	Sends logs to Amazon CloudWatch	AWS environments
gcplogs	Sends logs to Google Cloud Logging	GCP environments

2 How Logging Works



By default, Docker captures stdout and stderr of the container and sends it to the configured logging driver.

3 Configure Logging Driver

- Daemon-wide (in /etc/docker/daemon.json):

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

This sets default logging driver for all containers !

- Per-container (using docker run):

```
docker run -d \
  --log-driver=syslog \
  --log-opt syslog-address=udp://192.168.1.10:514
  --name myapp nginx
```

4 View Container Logs

- Use docker logs to view logs from a container.

```
docker logs <container_id>
docker logs -f <container_id>    # follow logs
docker logs --tail 100 <container_id>
docker logs --since 1h <container_id>
```

Logs are critical for troubleshooting !

5 docker stats – Real time Monitoring

- Shows real time resource usage of containers.

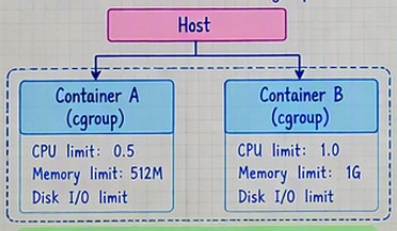
```
$ docker stats
CONTAINER ID   NAME   CPU %   MEM USAGE / LIMIT   MEM %   NET I/O     BLOCK I/O   PIDS
a1b2c3d4e5f6   web    1.23%   120MiB / 512MiB     23.4%   10MB / 5MB  1.2MB / 0B  12
f6e5d4c3b2a1   db     0.50%   300MiB / 1GiB       29.3%   2MB / 1MB   10MB / 5MB  25
```

Metrics shown:

- CPU % – CPU usage
- MEM USAGE/LIMIT – Memory used vs limit
- MEM % – Percentage of memory used
- NET I/O – Network input/output
- BLOCK I/O – Disk read/write
- PIDS – Number of processes

6 cgroups – Resource Control

- Docker uses Linux cgroups to limit and isolate resource usage (CPU, memory, I/O, etc.).
- Each container runs in its own cgroup.



cgroups ensure one container cannot consume all resources !

7 Memory & CPU Limits

- You can set resource limits when running a container.

Memory limit:

```
docker run -d --memory=512m \
              --memory-swap=1g  nginx
```

CPU limit:

```
docker run -d --cpus=0.5 nginx
# or
docker run -d --cpuset-cpus="0,1" nginx
```

If a container exceeds its limit:

- Memory: container may be OOM killed
- CPU: container is throttled (slower performance)

8 Log Rotation (json-file)

- Without log rotation, log files can grow and fill the disk.
- Use max-size and max-file options.

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",   # each file 10MB
    "max-file": "5"      # keep 5 files
  }
}
```

Always enable log rotation in production !

9 Common Scenarios & Solutions

Scenario 1: Logs Missing

- Check if container is still running.
- Verify logging driver (docker inspect).
- Application may be writing logs to file. instead of stdout/stderr.
- Check log rotation (older logs deleted).
- Ensure correct permissions for log driver.

Troubleshooting Commands:

```
docker ps
docker inspect <container_id> | grep -i log
docker logs <container_id>
```

Scenario 2: Container Uses Too Much Memory

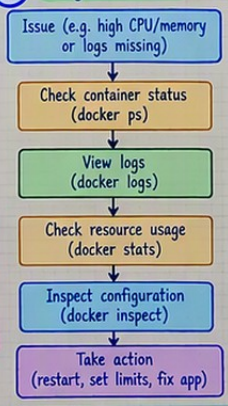
- Check memory usage: docker stats
- Verify memory limit (if not set).
- Application may have memory leak.
- Check container logs for OOMKilled.
- Consider setting memory limits and restart policy.

Useful Commands:

```
docker stats
docker inspect <container_id> | -i memory
dmesg | grep -i kill    # check OOM events
```

Monitor before it becomes a problem !

10 Diagnostic Flow



11 Interview Questions

- [x] What are Docker logging drivers?
- [x] What is the default logging driver?
- [x] How do you configure a logging driver for a container?
- [x] What is the difference between daemon-wide and per-container logging config?
- [x] How does docker logs work?
- [x] What metrics are shown in docker stats?
- [x] What are cgroups and why are they used?
- [x] How do you set memory and CPU limits?
- [x] Why is log rotation important?
- [x] How would you troubleshoot missing logs?
- [x] How do you identify a memory leak in a container?
- [x] Explain how you would monitor a container in production.

Logs help you debug today,
Monitoring helps you prevent issues tomorrow !

Observe → Understand → Optimize → Run Better !

Build Run Troubleshoot Be Ready!

Day 9: Docker Troubleshooting – Interview Gold

Real Problems ... Real Commands ... Real Solutions ... !

A calm mind and the right commands solve 90% problems!

1 Container cannot connect to DB

Symptoms:
- App inside container cannot
- connect to database
- Connection refused / timeout

Checks:
- docker ps
- docker logs <container>
- docker exec -it <container> sh
- ping <db-host>
- nslookup <db-host>
- docker network inspect <network>

Likely Cause:
- Wrong hostname / network
- DB not reachable / not running
- Firewall / security group
- Containers on different networks

Fix:
- Use correct hostname (service name)
- Connect containers to same network
- Check DB is running and accessible
- Verify ports and diffewall rules

2 Image build stuck

Use --progress=plain to see detailed build logs!

Symptoms:
- docker build hangs at a step
- No progress for a long time

Checks:
- docker build --progress=plain .
- Check which step is stuck
- Verify internet connectivity
- Check if fetching packages

Likely Cause:
- Large package download (apt, npm, pip)
- Network/DNS issues
- No cache, building from scratch
- Waiting for user input (e.g. tzdata)

Fix:
- Use cache: --cache-from or BuildKit
- Check network / proxy / DNS
- Use smaller base image
- Add -y to avoid interactive prompts
- Use multi-stage builds

3 Port already in use

Use docker ps and lsof to find what's using the port!

Symptoms:
- Error: bind: address already in use
- Container fails to start

Checks:
- sudo lsof -i :<port>
- sudo ss -tulnp | grep :<port>
- docker ps (check existing containers)
- netstat -tulnp | grep :<port>

Likely Cause:
- Another process/container using the port
- Port not released from previous run

Fix:
- Stop the conflicting service/container
- Use a different host port (e.g. -p 8081:80)
- Kill the process: sudo kill -9 <PID>
- For Docker: docker stop <container>

4 Permission denied writing files

Avoid running containers as root in production. Use correct permissions!

Symptoms:
- Error: Permission denied
- Cannot write to mounted volume

Checks:
- docker exec -it <container> id
- ls -l <path>
- docker inspect <container> | grep Mounts -A 5
- Check file/folder ownership on host

Likely Cause:
- Container running as non-root user
- Volume ownership mismatch
- Read-only filesystem

Fix:
- Run as root or correct user (USER in Dockerfile)
- Change ownership: chown -R 1000:1000 <path>
- Use :rw for volumes (not :ro)
- Set proper permissions on host

5 No space left on device

Use docker system df to see what's using space!

Symptoms:
- Error: No space left on device
- Container/image build fails

Checks:
- df -h
- docker system df
- du -sh /var/lib/docker
- docker ps -a
- docker images
- docker volume ls

Likely Cause:
- Docker images, containers, volumes consuming disk space
- Logs growing
- Dangling images not removed

Fix:
- Remove unused resources:
  - docker system prune -a
  - docker volume prune
- Clean logs: truncate or rotate
- Increase disk size if needed

6 ImagePullBackOff (Kubernetes)

Always verify the image name, tag and registry access!

Symptoms:
- Pods stuck in ImagePullBackOff
- Unable to pull Docker image

Checks:
- kubectl describe pod <pod-name>
- kubectl get events
- Check image name & tag
- Try pulling image manually: docker pull <image>

Likely Cause:
- Wrong image name/tag
- Image not available in registry
- Missing imagePullSecrets (private registry)
- Network/registry access issues

Fix:
- Use correct image name & tag
- Verify image exists and is accessible
- Configure imagePullSecrets for private registry
- Check network/DNS/proxy to registry

7 Cannot run docker without sudo

Docker socket gives root access. Only trusted users should be in docker group!

Symptoms:
- Permission denied while running docker
- Need to use sudo for docker commands

Checks:
- docker version
- ls -l /var/run/docker.sock
- id
- groups

Likely Cause:
- User not in docker group

Fix:
- Add user to docker group:
- sudo usermod -aG docker $USER
- Log out and log back in (or newgrp docker)
- Verify: groups

8 Container restarts again and again

Check logs first! Logs usually tell the real reason!

Symptoms:
- Container exits and restarts continuously
- docker ps shows Restarting

Checks:
- docker logs <container>
- docker inspect <container> | grep -A 5
- Check exit code: docker inspect -format='{{.State.ExitCode}}' <container>

Likely Cause:
- Application crash
- Incorrect CMD/ENTRYPOINT
- Missing environment variables / config
- Health check failing

Fix:
- Fix the application or configuration
- Use correct CMD/ENTRYPOINT
- Check required environment variables
- Test container interactively: docker run -it <image> sh

9 Application not accessible

Running ≠ Reachable! Check ports, networks and firewall!

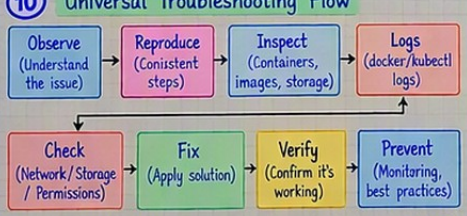
Symptoms:
- Container is running but application
- not reachable from browser/other container

Checks:
- docker ps (check port mapping)
- docker inspect <container> NetworkSettings
- curl http://localhost:<port>
- docker exec -it <container> curl localhost:<port>
- Check firewall/security group

Likely Cause:
- Wrong port mapping
- App listening on different port or localhost only
- Firewall blocking access
- Container on different network

Fix:
- Map correct port: -p <host>:<container>
- Ensure app listens on 0.0.0.0 (not 127.0.0.1)
- Check network and firewall rules
- Use docker logs to verify app started properly

10 Universal Troubleshooting Flow



"Systematic troubleshooting saves time and frustration!"

11 Interview Gold (Be Ready!)

- ✅ How do you troubleshoot a container that keeps restarting?
- ✅ What steps do you take when a container cannot connect to a DB?
- ✅ How do you check what is using a port?
- ✅ How do you reduce Docker image size?
- ✅ What is ImagePullBackOff and how to fix it?
- ✅ How do you allow a non-root user to run docker?
- ✅ How do you check disk usage in Docker?
- ✅ What commands do you use to debug networking issues?
- ✅ How do you read container logs and find the root cause?

12 Important Tips & Cautions

- ✅ Avoid using --privileged unless needed.
- ✅ Be careful with docker system prune -a (to avoid deleting important data).
- ✅ Do not expose Docker socket to containers.
- ✅ Use specific tags (not latest) in production.
- ✅ Monitor disk usage, logs and resources regularly.
- ✅ Keep images small and secure.
- ✅ Use health checks and proper restart policies.
- ✅ Verify network and firewall settings.
- ✅ Follow least privilege principle.

"Learn Practice Troubleshoot Grow!